| Ex No 3 | Simple GAN with TensorFlow and MNIST Dataset |
|---|---|

**Aim**

To implement and understand a simple Generative Adversarial Network (GAN) using TensorFlow on the MNIST handwritten digit dataset and generate synthetic digit images.

**Objectives**

- Understand the working of Generator and Discriminator in GAN.
- Learn how GANs generate synthetic images from random noise.
- Implement a simple GAN using TensorFlow/Keras.
- Visualize the generated handwritten digit images.

**Algorithm**

1. Import required TensorFlow, Keras, NumPy and Matplotlib libraries.
2. Load the MNIST dataset using keras.datasets.
3. Preprocess the images by converting them to float values and normalizing them.
4. Reshape the dataset to include the channel dimension.
5. Define training parameters such as batch size, noise dimension and number of epochs.
6. Create dataset batches using TensorFlow Dataset API.
7. Build the Generator model to convert random noise into image data.
8. Build the Discriminator model to classify images as real or fake.
9. Define loss function and optimizers for training.
10. Train the Discriminator using real images and generated fake images.
11. Train the Generator to fool the Discriminator.
12. Repeat the process for multiple epochs.
13. Generate synthetic digit images after training and display them.

**Code Walkthrough with Explanation**

**1. Import Required Libraries**

Code:

```
import tensorflow as tf

from tensorflow import keras

import numpy as np

import matplotlib.pyplot as plt
```

Explanation:

This section imports the libraries required for the experiment.
TensorFlow and Keras are used to build and train neural networks.

NumPy helps perform numerical operations, and Matplotlib is used to visualize the generated images.

**2. Load the MNIST Dataset**

Code:

```
(x_train, _), (_, _) = keras.datasets.mnist.load_data()
```

Explanation:

This line loads the MNIST handwritten digit dataset.
Only the images are required for GAN training, so the labels are ignored.

**3. Preprocess the Images**

Code:

```
x_train = x_train.astype("float32")

x_train = (x_train - 127.5) / 127.5   # Normalize to [-1, 1]

x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)
```

Explanation:

The images are converted to float format for computation.
Pixel values are normalized to the range [-1,1] because the generator uses tanh activation.
Images are reshaped to include the channel dimension (28×28×1).

**4. Define Training Parameters**

Code:

```
batch_size = 64
noise_dim = 100
epochs = 10
```

Explanation:

Batch size determines how many images are processed in one step.
Noise dimension represents the size of the random vector given to the generator.
Epochs represent how many times the dataset is used for training.

**5. Create Dataset Batches**

Code:

```
dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000).batch(batch_size)
```

Explanation:

TensorFlow dataset API is used to create shuffled batches of the dataset.
This improves training efficiency and prevents the model from learning patterns in data order.

**6. Build the Generator**

Code:

```
generator = keras.Sequential([
    keras.layers.Input(shape=(noise_dim,)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(28 * 28 * 1, activation='tanh'),
    keras.layers.Reshape((28, 28, 1))
```

Explanation:

The generator creates fake images from random noise.
The Dense layers learn patterns from the noise and produce 784 pixel values.
The reshape layer converts this vector into a 28×28 image.

**7. Build the Discriminator**

Code:

```
discriminator = keras.Sequential([
    keras.layers.Input(shape=(28, 28, 1)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
```

Explanation:

The discriminator evaluates whether an image is real or fake.
Flatten converts the image into a vector and dense layers classify it.
The sigmoid output gives a probability value between 0 and 1.

**8. Define Loss and Optimizers**

Code:

```
loss_fn = keras.losses.BinaryCrossentropy()
g_optimizer = keras.optimizers.Adam(learning_rate=0.0002)
d_optimizer = keras.optimizers.Adam(learning_rate=0.0002)
```

Explanation:

Binary cross entropy is used because this is a binary classification problem.
Adam optimizer updates the parameters of Generator and Discriminator.

**9. Training Process**

Start training for multiple epochs

for epoch in range(epochs):

Explanation

- This loop runs the training process for the specified number of epochs.
- One epoch means the model sees the entire training dataset once.
- If epochs = 20, the GAN will train on the full dataset 20 times.

**Read one batch of real images**

for real_images in dataset:

Explanation

- The dataset is divided into batches.
- In each iteration, one batch of real MNIST images is taken.
- This helps training happen in smaller chunks instead of loading all images at once.

**Find the current batch size**

batch_size_now = real_images.shape[0]

Explanation

- This gives the number of images in the current batch.
- Usually it is equal to the chosen batch size, such as 64.
- The last batch may sometimes contain fewer images.

**Create labels for real and fake images**

real_labels = tf.ones((batch_size_now, 1))
fake_labels = tf.zeros((batch_size_now, 1))

Explanation

- real_labels contains all 1s, which means “real image”.
- fake_labels contains all 0s, which means “fake image”.
- These labels are used while training the Discriminator.

**Train Discriminator**

Generate random noise

noise = tf.random.normal((batch_size_now, noise_dim))

Explanation

- Random numbers are created from a normal distribution.
- These random values are the input to the Generator.
- The size of the noise vector is noise_dim.

---

**Generator creates fake images**

fake_images = generator(noise, training=True)

Explanation

- The Generator takes the random noise as input.
- It converts that noise into fake handwritten digit images.
- These images are not real; they are synthetic images created by the Generator.

---

**Start recording Discriminator operations**

with tf.GradientTape() as tape:

Explanation

- GradientTape records all computations inside this block.
- TensorFlow uses this recorded information to compute gradients later.
- These gradients are needed to update the Discriminator weights.

---

**Discriminator checks real images**

real_output = discriminator(real_images, training=True)

Explanation

- The Discriminator receives real images from the dataset.
- It predicts whether each image is real.
- The output values are probabilities close to 1 for real images.

---

**Discriminator checks fake images**

fake_output = discriminator(fake_images, training=True)

Explanation

- Now the Discriminator receives fake images made by the Generator.
- It predicts whether each image is fake.
- The output values should ideally be close to 0 for fake images.

---

**Compute loss for real images**

real_loss = loss_fn(real_labels, real_output)

Explanation

- This compares the Discriminator's prediction for real images with the correct label 1.
- If the Discriminator correctly identifies real images, the loss becomes smaller.

---

**Compute loss for fake images**

fake_loss = loss_fn(fake_labels, fake_output)

Explanation

- This compares the Discriminator's prediction for fake images with the correct label 0.
- If the Discriminator correctly identifies fake images, this loss becomes smaller.

---

**Total Discriminator loss**

d_loss = real_loss + fake_loss

Explanation

- The total Discriminator loss is the sum of:
    - real image loss
    - fake image loss
- This tells how well the Discriminator is performing overall.

---

**Compute Discriminator gradients**

d_gradients = tape.gradient(d_loss, discriminator.trainable_variables)

Explanation

- TensorFlow calculates how the Discriminator weights should change to reduce the loss.
- These changes are called gradients.

---

**Update Discriminator weights**

d_optimizer.apply_gradients(zip(d_gradients, discriminator.trainable_variables))

Explanation

- The optimizer applies the gradients to update the Discriminator's weights.
- After this step, the Discriminator becomes better at distinguishing real and fake images.

---

**Train Generator**

Create new random noise

noise = tf.random.normal((batch_size_now, noise_dim))

Explanation

- A new set of random noise vectors is created.
- This is used again as input to the Generator for its own training step.

---

**Start recording Generator operations**

with tf.GradientTape() as tape:

Explanation

- TensorFlow again records all operations inside this block.
- This time it will compute gradients for the Generator.

---

**Generator creates new fake images**

generated_images = generator(noise, training=True)

Explanation

- The Generator creates another batch of fake images from the noise.
- These images will now be used to train the Generator.

---

**Discriminator evaluates generated images**

output = discriminator(generated_images, training=True)

Explanation

- The generated images are passed to the Discriminator.
- The Discriminator tries to judge whether they are real or fake.

---

**Compute Generator loss**

g_loss = loss_fn(real_labels, output)

Explanation

- This is the important GAN idea.
- The Generator wants the Discriminator to think fake images are real.
- So the generated images are compared with real_labels instead of fake_labels.

Meaning

- If the Discriminator says the fake image is real, the Generator succeeds.
- If not, the Generator must improve.

---

**Compute Generator gradients**

g_gradients = tape.gradient(g_loss, generator.trainable_variables)

Explanation

- TensorFlow computes how the Generator weights should change to reduce Generator loss.
- These gradients help the Generator learn to create better fake images.

**Update Generator weights**

g_optimizer.apply_gradients(zip(g_gradients, generator.trainable_variables))

Explanation

- The optimizer updates the Generator's weights.
- After this step, the Generator improves and produces more realistic images.

**End of One Epoch**

```
print(f"Epoch [{epoch+1}/{epochs}] D Loss: {d_loss.numpy():.4f} G Loss: {g_loss.numpy():.4f}")
```

Explanation

- After all batches in one epoch are completed, the loss values are printed.
- D Loss shows how well the Discriminator is working.
- G Loss shows how well the Generator is learning to fool the Discriminator.

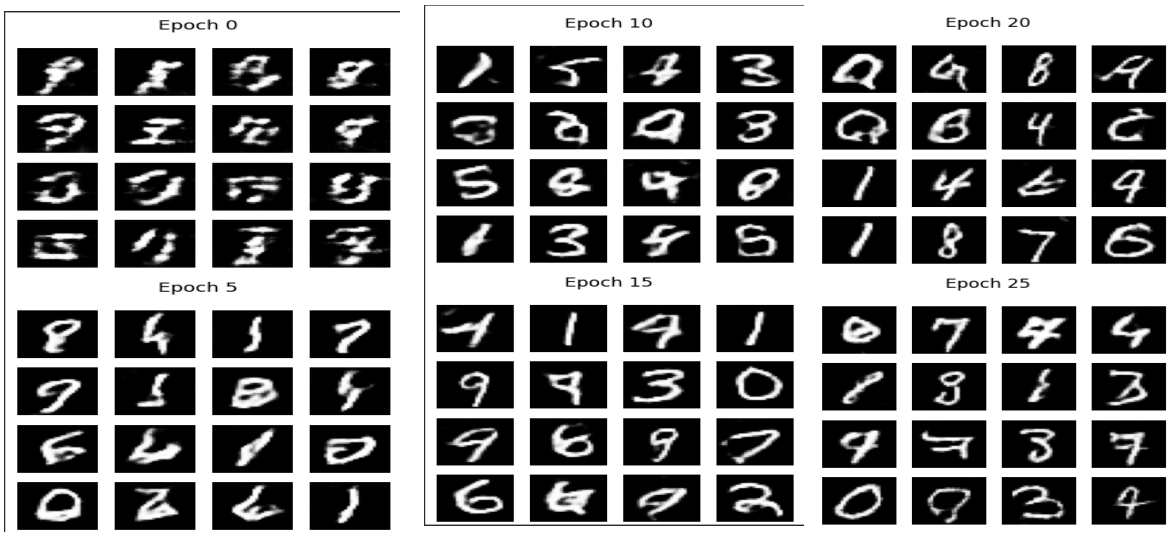
**10.Generate images after training**

```
noise = tf.random.normal((16, noise_dim))
generated_images = generator(noise, training=False)
```

**11. Display generated images**

```
fig, axes = plt.subplots(4, 4, figsize=(6, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow((generated_images[i, :, :, 0] + 1) / 2, cmap='gray')
    ax.axis("off")
plt.tight_layout()
plt.show()
```

**Output:**



| Question No | Question | Student Answer |
|---|---|---|
| 1 | **Identify the section of the code where the MNIST dataset is loaded. Why are the labels ignored in this GAN implementation?** | The MNIST dataset is loaded using:<br>(x_train, _), (_, _) = keras.datasets.mnist.load_data()<br>Labels are ignored because GANs do not perform classification; they only generate and distinguish real vs fake images. |
| 2 | **Observe the preprocessing step (x_train - 127.5) / 127.5. Why are the pixel values converted to the range [-1, 1]?** | Pixel values are converted to [-1, 1] to match the tanh activation used in the Generator, which improves training stability. |
| 3 | **Locate the variable noise_dim = 100. What is the purpose of the random noise vector in the Generator?** | The random noise vector (noise_dim = 100) is the input to the Generator. It helps generate diverse synthetic images. |
| 4 | **Identify the Generator architecture in the code. How many Dense layers are used in the Generator model?** | The Generator has 2 Dense layers:<br>• First: 128 neurons<br>• Second: 784 neurons (28×28×1) |
| 5 | **Locate the Discriminator model in the program. What is the purpose of the Flatten layer?** | The Flatten layer converts a 2D image into a 1D vector so it can be processed by Dense layers. |
| 6 | **Observe the final layer of the Discriminator that uses sigmoid activation. Why is sigmoid used here?** | Sigmoid activation is used to output a probability between 0 and 1, indicating real or fake. |
| 7 | **Identify the loss function used in the GAN. Why is Binary Cross Entropy used in this experiment?** | The loss function is Binary Cross Entropy, used because the task is binary classification (real vs fake). |

| Question No | Question | Student Answer |
|---|---|---|
| **8** | **Why is the Discriminator trained using both real images and generated fake images?** | The Discriminator is trained on both:<br>• Real images → labeled 1<br>• Fake images → labeled 0<br>This helps it learn to distinguish properly. |
| **9** | **In Generator training, fake images are compared with real_labels. Why is this done?** | Fake images are compared with real_labels so the Generator learns to fool the Discriminator**.** |
| **10** | **How many synthetic images are displayed at the end of training?** | A total of 16 synthetic images are displayed (4 × 4 grid). |
| **11** | **Change epochs = 20 to epochs = 50 and run the program again. What difference do you observe in the generated images?** | Increasing epochs to 50 results in:<br>• More realistic and clearer digits<br>• Better Generator performance |
| **12** | **Change noise_dim from 100 to 50. What happens to the generated images?** | Reducing noise_dim to 50:<br>• Reduces variation<br>• Images may become less detailed |
| **13** | **Increase the number of neurons in the Generator from 128 to 256. Does the output improve?** | Increasing neurons to 256:<br>• Can improve image quality<br>• But may increase training time |
| **14** | **Change the batch size from 64 to 128. What impact does this have on training?** | Increasing batch size to 128:<br>• Faster training per epoch<br>• May reduce learning stability slightly |
| **15** | **Add one more Dense layer to the Generator. Does the quality of images improve?** | Adding another Dense layer:<br>• Improves learning capacity<br>• May produce better images if tuned properly |

| Question No | Question | Student Answer |
|---|---|---|
| **16** | **Add one more Dense layer to the Discriminator. What change do you observe in training results?** | Adding a Dense layer to Discriminator:<br>• Makes it stronger<br>• Can improve classification but may overpower Generator |
| **17** | **Replace ReLU with LeakyReLU in the networks. Does training become more stable?** | Using LeakyReLU:<br>• Prevents dead neurons<br>• Makes training more stable |
| **18** | **Observe Generator loss and Discriminator loss during training. What trend do you notice?** | Trend observed:<br>• Discriminator loss fluctuates<br>• Generator loss decreases gradually<br>• Both compete with each other |
| **19** | **Compare the results of the original GAN and improved GAN. Which one produces clearer digits?** | Improved GAN produces:<br>• Clearer<br>• More realistic digits<br>Compared to the original |
| **20** | **Why does the Generator initially produce random noisy images?** | Initially, Generator produces noisy images because:<br>• It starts with random weights<br>• It has not learned patterns yet |
| **21** | **What happens if the Discriminator becomes too strong compared to the Generator?** | If Discriminator becomes too strong:<br>• Generator cannot learn<br>• Training may fail |
| **22** | **Why is GAN training considered unstable compared to normal neural networks?** | GAN training is unstable because:<br>• Two models compete<br>• Balance between them is difficult<br>• Small changes affect both networks |

| Question No | Question | Student Answer |
|---|---|---|
| **23** | **Mention two real-world applications of GANs other than image generation.** | Two real-world applications:<br>• Image super-resolution<br>• Deepfake generation / face synthesis |
| **24** | **How can this GAN architecture be modified to generate color images instead of grayscale images?** | To generate color images:<br>• Change output shape to **(height, width, 3)**<br>• Modify Discriminator input<br>• Use RGB datasets like CIFAR-10<br>• Normalize all channels |

**Result**

Thus, a simple Generative Adversarial Network (GAN) was implemented using TensorFlow and trained on the MNIST dataset. The Generator successfully produced synthetic handwritten digit images similar to the original MNIST digits.